

RAG Assignment

Section A

1. What is a Vector Database?

A Vector Database is used to store and search vectors (embeddings), where embeddings are the numerical representation of data. It is different from a normal database such as MySQL. MySQL stores structured data and is generally used for getting an exact match for a query, whereas a vector database searches for vectors which are semantically similar.

2. Explain the Steps from Raw Text to Vector Database

Step 1 – Raw Text

Raw text contains the text from the document which humans can read.

Step 2 – Parsing

Parsing means extracting the text from the document.

For a small sentence, no complex parsing is required.

Step 3 – Chunking

If the content comes from a large document, we divide the document into smaller chunks for better processing.

For example, our sentence can be kept in one chunk or two chunks.

Chunk 1 - AI is taking a lot of jobs nowadays.

Step 4 – Embedding

Embedding means converting our text into numerical embeddings.

For example:

AI is taking a lot of jobs nowadays. $\rightarrow (0.88, 0.645, 0.99, \dots)$

Step 5 – Vector Database

The embeddings are stored in the vector database.

3. Exact Lookup vs Similarity Search

A B-tree is designed mainly for finding exact or ordered values, which is called exact lookup.

However, embeddings have numerical values. When a user gives a query vector, we do not want an exact match. Instead, we want to find which vectors are closest to our search. This is called similarity search.

Example:

- Exact lookup – Find data of ID 100.
- Similarity search – Find the vectors similar to this vector.

4. Cosine Similarity

Given:

$$A = [1, 2, 3]$$

$$B = [2, 4, 6]$$

Cosine similarity is:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Step 1 – Dot Product

$$A \cdot B = (1)(2) + (2)(4) + (3)(6)$$

$$= 2 + 8 + 18$$

$$= 28$$

Step 2 – Magnitude of A

$$\|A\| = \sqrt{1^2 + 2^2 + 3^2}$$

$$= \sqrt{1 + 4 + 9}$$

$$= \sqrt{14}$$

Step 3 – Magnitude of B

$$\|B\| = \sqrt{2^2 + 4^2 + 6^2}$$

$$= \sqrt{4 + 16 + 36}$$

$$= \sqrt{56}$$

Step 4 – Cosine Similarity

$$\cos(\theta) = \frac{28}{\sqrt{14}\sqrt{56}}$$

Since:

$$\sqrt{56} = 2\sqrt{14}$$

we get:

$$\cos(\theta) = \frac{28}{\sqrt{14}(2\sqrt{14})}$$

$$\begin{aligned}
&= \frac{28}{2(14)} \\
&= \frac{28}{28} \\
&= 1
\end{aligned}$$

Therefore, the cosine similarity is:

$$\boxed{1}$$

The cosine similarity is 1, meaning that the two vectors point in exactly the same direction.

This happens because:

$$B = 2A$$

Therefore, the vectors have the same direction even though their magnitudes are different.

5. Why is a Vector Database Essential for RAG?

A vector database is an important part of a RAG system because it stores the embeddings of documents and helps retrieve the most relevant information for a user's query.

When a user asks a question, the question is converted into an embedding. The vector database compares this query vector with the stored document vectors and retrieves the most similar chunks. These retrieved chunks are then provided to the LLM as context to generate the answer.

Without the vector database, the RAG system would not have an efficient way to search for the required information.

Section C

C1. Multimodal RAG

I would use Multimodal RAG because the product manual contains both text and images.

The system would store the text, diagrams, and photos and retrieve the relevant information when the user asks a question.

For example, if the answer to "How do I attach the side panel?" is present only in a diagram, the system would retrieve that diagram and give it to a multimodal LLM to understand and answer the question.

A text-only RAG system may fail because it cannot properly retrieve or understand information that exists only inside the diagram.

C2. Unified Embedding Space vs Separate Indexes + Fusion

Unified embedding space converts both text and images into embeddings in the same space. This makes the retrieval process simpler because both can be searched together.

Separate indexes + fusion stores text and images separately. The system searches both indexes and then combines their results.

The main difference is that unified embedding uses one common space, while separate indexes use different searches and combine the results.

We will use separate indexes for a product manual because the manual contains different types of information, such as text, diagrams, and photos. Text and images can be processed and searched using different methods. A text index can efficiently search written instructions, while an image index can search diagrams and photos. Their results can then be combined using fusion.